

Project 0 — Bake-Off: Text-to-SQL

CS496 AI Engineering, Fall 2026. Eval run **2026-09-23**.

1. The task

Given a natural-language question and a fixed schema, each model makes **one call** and returns a single SQLite `SELECT`. Scoring is execution accuracy: the generated SQL and the gold SQL both run against the same seeded retail database (5 tables — `categories`, `customers`, `products`, `orders`, `order_items`) and their result sets are compared order-insensitively with floats rounded to 2dp (`src/db_utils.py::rows_match`). No LLM judge, no human in the scoring loop. A parse error, a rejected statement, or an API failure all count as wrong.

Example item (`data/items.jsonl`, id 1, easy):

Q: List the names of all customers from Tunisia, ordered alphabetically by name.
gold: `SELECT name FROM customers WHERE country = 'Tunisia' ORDER BY name;`

2. The data

50 items, hand-written against `db/schema.sql` — not an external dataset, so the correct answer is unambiguous and re-derivable from a clean clone. The database is seeded deterministically (`random.seed(42)`): 30 customers, 8 categories, 40 products, 160 orders, ~407 order items).

Labels were checked mechanically: every question/SQL pair lives in `data/make_items.py`, which **executes each gold query against the freshly built database before writing `items.jsonl`**. A SQL error aborts generation, so no item can reach the dataset with a typo or a bad join. A second read-through checked that each question and its SQL express the same intent. Two items legitimately return zero rows (customers who never ordered; products never ordered) — under this seed, every customer and product has at least one order. Difficulty split: 15 easy, 19 medium, 16 hard. **No dev/test split** — all 50 items are the test set, per the brief. See `data/LABELLING.md`.

3. Setup

Role	Model ID	Served by
top API	<code>openai/gpt-oss-120b</code>	Groq
cheap API	<code>openai/gpt-oss-20b</code>	Groq
open-weights	<code>qwen2.5-coder:7b-instruct (Q4)</code>	Ollama 0.34.2, local

Settings, identical across all three: temperature 0, `max_tokens=512`, one call per item, calls issued sequentially (not concurrently) so the latency numbers are not distorted by contention. No retrieval, no agent, no few-shot examples. The prompt is byte-for-byte identical across models (`src/prompt.py`) — the schema is pasted in as text, and the model is told to emit only SQL, no fences.

Two things to flag. **Both API-tier models are the same family at two sizes**, not two different labs — so this measures "does model size matter" more than "does provider matter". We moved to Groq after Gemini's free tier capped at 5 requests/minute and returned `429 RESOURCE_EXHAUSTED` on 96% of a first run; Groq's free tier allows 30/minute. **Cost is computed at published list price even though actual spend was zero** on the free tier, because a \$0 figure makes the 100x projection meaningless. Hardware for the self-hosted model: `results/HARDWARE.md`.

4. Results

Zero call errors on all three models; all 150 calls returned.

Model	Accuracy	Easy	Medium	Hard	p50	p95	Cost/1k
<code>gpt-oss-120b</code>	50/50 (100%)	15/15	19/19	16/16	972 ms	5,445 ms	\$0.136
<code>gpt-oss-20b</code>	48/50 (96%)	15/15	18/19	15/16	1,055 ms	7,443 ms	\$0.086
<code>qwen2.5-coder:7b</code>	47/50 (94%)	15/15	18/19	14/16	14,136 ms	25,152 ms	\$0.057 [†]

[†] at 100% utilisation — see section 6.

One caveat on the latency column: `call_groq` starts its timer before the Groq SDK's internal retries, so a 429 backoff is counted as model latency. One call (`gpt-oss-120b` item 13) took 63.9 s to emit 73 tokens and is almost certainly a retry rather than generation. Both such outliers sit above p95, so the p50/p95 figures above are unaffected, but mean latency for the 120B is inflated by roughly a second. See `postmortem.md`.

Every model solved every easy item. Only 5 misses exist in total, so the brief's "3 wrong answers per model" is capped by the data: `gpt-oss-120b` **had none**.

`gpt-oss-20b` (2 misses)

- item 26 (medium)* — "5 products with the highest total quantity sold". Added an unrequested `WHERE o.status = 'completed'` and joined `orders` to do it. Well-formed SQL answering a narrower question.
- item 45 (hard)* — "customers who spent more than 300 on completed orders". **Returned an empty string**. Not a SQL failure: `output_tokens = 512`, exactly the cap. The model spent its entire budget on reasoning tokens before emitting any answer. The 20B reasons more verbosely than the 120B (205 vs 144 average output tokens), so it hits the ceiling first. A larger `MAX_OUTPUT_TOKENS` would likely recover this — a harness limitation, not a capability gap.

qwen2.5-coder:7b (3 misses)

- *item 31 (medium)* — most recent order per customer. Added `WHERE o.status = 'completed'`; also grouped by `customer_id` where the gold grouped by `name`.
- *item 46 (hard)* — distinct customers per category. Same unrequested `completed` filter.
- *item 50 (hard)* — average order value. A real aggregation error: returned `SUM(quantity * unit_price)` across all completed orders where the question asked for the **average per order**, missing the `GROUP BY order_id` subquery. This is the only one of the five misses that reflects a genuine failure to reason about SQL semantics.

The dominant failure mode is not SQL syntax — it is scope. Three of the five misses are the same mistake: silently narrowing the question with `WHERE status = 'completed'`. That is arguably as much a prompt specification gap as a model error, and it is the highest-leverage thing to fix — stating explicitly whether order status should filter results would likely close most of the gap between all three models.

Determinism caveat. Temperature 0 did not produce byte-identical output across runs. Re-running the Groq models a day apart changed the generated SQL on 14/50 (120B) and 8/50 (20B) items, with no change in accuracy. This is expected for batched mixture-of-experts serving, but it means **the 50 vs 48 gap is within run-to-run noise** and should not be read as clean separation between the two API models.

5. Our choice, and when we would change it

We would ship **openai/gpt-oss-120b on Groq**. It was perfect on all 50 items, it has the lowest p50 latency of the three (972 ms), and at \$0.136 per 1,000 requests the cost is negligible — buying the accuracy margin over the cheap model costs about five cents per thousand queries, which is not a trade worth making for a user-facing feature. The honest version of that claim: given the determinism caveat above, we can say the 120B is *at least as good* as the 20B, not that it is reliably better.

Three conditions would flip the choice. **Privacy** is the strongest: if schemas or user questions could not leave our infrastructure, `qwen7b` becomes the only option, and 94% for a 7B running partly on CPU is a genuinely good showing. **Volume** in the 106–253 requests/hour band makes self-hosting cheaper per request (section 6) — though its 14 s p50 rules it out for anything interactive, so this only applies to batch workloads. **Accuracy floor:** if the task demanded better than 100% on a harder question set, none of these one-call setups would qualify, and we would add schema retrieval, self-consistency sampling, or a validate-and-retry loop before changing model.

6. Cost at 100x traffic and break-even volume

API cost per request is flat regardless of volume — Groq's per-token pricing has no bulk discount modeled, so **100x traffic is exactly 100x the cost:** \$13.60 per 100k requests for the 120B, \$8.63 for the 20B. No projection needed.

The self-hosted number is the one that needs care. The \$0.057/1k figure is electricity only (\$0.0144/hour: ~120 W sustained at ~USD 0.12/kWh) and assumes the laptop runs requests **back-to-back at 100% utilisation** — at a 14.2 s average, ~253 requests/hour. That is a throughput ceiling, not a typical load. Because the hourly cost is fixed whether the box is busy or idle, cost per request falls as volume rises until it reaches that ceiling:

- Below **~106 req/hour**, the 120B API is cheaper per request.
- Below **~167 req/hour**, the 20B API is cheaper per request.
- Above **~253 req/hour**, the laptop cannot keep up at all; you need a second box, and cost per 1k goes flat again.

So self-hosting wins only inside a narrow **106–253 req/hour window**, and only for workloads that tolerate 14-second latency. Our expected traffic is well below 106 req/hour, which puts us firmly in the regime where the API is both cheaper and ~14x faster. Two assumptions make this figure favourable to self-hosting and should be stated plainly: the hourly rate **excludes hardware amortisation and our own time**, and it prices a laptop we already own. Including amortisation would push the break-even volume substantially higher.